



Chapter 2. Programming Tools

Table of Contents

- 2.1. Synopsis
- 2.2. Introduction
- 2.3. Introduction to Programming
- 2.4. Compiling with `cc`
- 2.5. Make
- 2.6. Debugging
- 2.7. Using Emacs as a Development Environment
- 2.8. Further Reading

2.1. Synopsis

This chapter is an introduction to using some of the programming tools supplied with FreeBSD, although much of it will be applicable to many other versions of UNIX®. It does *not* attempt to describe coding in any detail. Most of the chapter assumes little or no previous programming knowledge, although it is hoped that most programmers will find something of value in it.

2.2. Introduction

FreeBSD offers an excellent development environment. Compilers for C and C++ and an assembler come with the basic system, not to mention classic UNIX® tools such as `sed` and `awk`. If that is not enough, there are many more compilers and interpreters in the Ports collection. The following section, [Introduction to Programming](#), lists some of the available options. FreeBSD is very compatible with standards such as POSIX® and ANSI C, as well with its own BSD heritage, so it is possible to write applications that will compile and run with little or no modification on a wide range of platforms.

However, all this power can be rather overwhelming at first if you have never written programs on a UNIX® platform before. This document aims to help you get up and



running, without getting too deeply into more advanced topics. The intention is that this document should give you enough of the basics to be able to make some sense of the documentation.

Most of the document requires little or no knowledge of programming, although it does assume a basic competence with using UNIX® and a willingness to learn!

2.3. Introduction to Programming

A program is a set of instructions that tell the computer to do various things; sometimes the instruction it has to perform depends on what happened when it performed a previous instruction. This section gives an overview of the two main ways in which you can give these instructions, or "commands" as they are usually called. One way uses an *interpreter*, the other a *compiler*. As human languages are too difficult for a computer to understand in an unambiguous way, commands are usually written in one or other languages specially designed for the purpose.

2.3.1. Interpreters

With an interpreter, the language comes as an environment, where you type in commands at a prompt and the environment executes them for you. For more complicated programs, you can type the commands into a file and get the interpreter to load the file and execute the commands in it. If anything goes wrong, many interpreters will drop you into a debugger to help you track down the problem.

The advantage of this is that you can see the results of your commands immediately, and mistakes can be corrected readily. The biggest disadvantage comes when you want to share your programs with someone. They must have the same interpreter, or you must have some way of giving it to them, and they need to understand how to use it. Also users may not appreciate being thrown into a debugger if they press the wrong key! From a performance point of view, interpreters can use up a lot of memory, and generally do not generate code as efficiently as compilers.

In my opinion, interpreted languages are the best way to start if you have not done any programming before. This kind of environment is typically found with languages like Lisp, Smalltalk, Perl and Basic. It could also be argued that the UNIX® shell (`sh` , `csh`) is itself an interpreter, and many people do in fact write shell "scripts" to help with various "housekeeping" tasks on their machine. Indeed, part of the original UNIX® philosophy was to provide lots of small utility programs that could be linked together in shell scripts to perform useful tasks.

2.3.2. Interpreters Available with FreeBSD



Here is a list of interpreters that are available from the FreeBSD Ports Collection, with a brief discussion of some of the more popular interpreted languages.

Instructions on how to get and install applications from the Ports Collection can be found in the [Ports section](#) of the handbook.

BASIC

Short for Beginner's All-purpose Symbolic Instruction Code. Developed in the 1950s for teaching University students to program and provided with every self-respecting personal computer in the 1980s, BASIC has been the first programming language for many programmers. It is also the foundation for Visual Basic.

The Bywater Basic Interpreter can be found in the Ports Collection as [lang/bwbasic](#) and the Phil Cockroft's Basic Interpreter (formerly Rabbit Basic) is available as [lang/pbasic](#).

Lisp

A language that was developed in the late 1950s as an alternative to the "number-crunching" languages that were popular at the time. Instead of being based on numbers, Lisp is based on lists; in fact, the name is short for "List Processing". It is very popular in AI (Artificial Intelligence) circles.

Lisp is an extremely powerful and sophisticated language, but can be rather large and unwieldy.

Various implementations of Lisp that can run on UNIX® systems are available in the Ports Collection for FreeBSD. GNU Common Lisp can be found as [lang/gcl](#). CLISP by Bruno Haible and Michael Stoll is available as [lang/clisp](#). For CMUCL, which includes a highly-optimizing compiler too, or simpler Lisp implementations like SLisp, which implements most of the Common Lisp constructs in a few hundred lines of C code, [lang/cmucl](#) and [lang/slisp](#) are available respectively.

Perl

Very popular with system administrators for writing scripts; also often used on World Wide Web servers for writing CGI scripts.

Perl is available in the Ports Collection as [lang/perl5.24](#) for all FreeBSD releases.

Scheme

A dialect of Lisp that is rather more compact and cleaner than Common Lisp. Popular in Universities as it is simple enough to teach to undergraduates as a first language, while it has a high enough level of abstraction to be used in research work.



Scheme is available from the Ports Collection as [lang/elk](#) for the Elk Scheme Interpreter. The MIT Scheme Interpreter can be found in [lang/mit-scheme](#) and the SCM Scheme Interpreter in [lang/scm](#).

Icon

Icon is a high-level language with extensive facilities for processing strings and structures. The version of Icon for FreeBSD can be found in the Ports Collection as [lang/icon](#).

Logo

Logo is a language that is easy to learn, and has been used as an introductory programming language in various courses. It is an excellent tool to work with when teaching programming to smaller age groups, as it makes creation of elaborate geometric shapes an easy task.

The latest version of Logo for FreeBSD is available from the Ports Collection in [lang/logo](#).

Python

Python is an Object-Oriented, interpreted language. Its advocates argue that it is one of the best languages to start programming with, since it is relatively easy to start with, but is not limited in comparison to other popular interpreted languages that are used for the development of large, complex applications (Perl and Tcl are two other languages that are popular for such tasks).

The latest version of Python is available from the Ports Collection in [lang/python](#).

Ruby

Ruby is an interpreter, pure object-oriented programming language. It has become widely popular because of its easy to understand syntax, flexibility when writing code, and the ability to easily develop and maintain large, complex programs.

Ruby is available from the Ports Collection as [lang/ruby32](#).

Tcl and Tk

Tcl is an embeddable, interpreted language, that has become widely used and became popular mostly because of its portability to many platforms. It can be used both for quickly writing small, prototype applications, or (when combined with Tk, a GUI toolkit) fully-fledged, featureful programs.

Various versions of Tcl are available as ports for FreeBSD. The latest version, Tcl 8.5, can be found in [lang/tcl87](#).



2.3.3. Compilers

Compilers are rather different. First of all, you write your code in a file (or files) using an editor. You then run the compiler and see if it accepts your program. If it did not compile, grit your teeth and go back to the editor; if it did compile and gave you a program, you can run it either at a shell command prompt or in a debugger to see if it works properly. [1]

Obviously, this is not quite as direct as using an interpreter. However it allows you to do a lot of things which are very difficult or even impossible with an interpreter, such as writing code which interacts closely with the operating system-or even writing your own operating system! It is also useful if you need to write very efficient code, as the compiler can take its time and optimize the code, which would not be acceptable in an interpreter. Moreover, distributing a program written for a compiler is usually more straightforward than one written for an interpreter-you can just give them a copy of the executable, assuming they have the same operating system as you.

As the edit-compile-run-debug cycle is rather tedious when using separate programs, many commercial compiler makers have produced Integrated Development Environments (IDEs for short). FreeBSD does not include an IDE in the base system, but [devel/kdevelop](#) is available in the Ports Collection and many use Emacs for this purpose. Using Emacs as an IDE is discussed in [Using Emacs as a Development Environment](#).

2.4. Compiling with cc

This section deals with the clang compiler for C and C++, as it's installed with the FreeBSD base system. Starting with FreeBSD 10.X clang is installed as `cc`; the GNU compiler [gcc](#) is available in the Ports Collection. The details of producing a program with an interpreter vary considerably between interpreters, and are usually well covered in the documentation and on-line help for the interpreter.

Once you have written your masterpiece, the next step is to convert it into something that will (hopefully!) run on FreeBSD. This usually involves several steps, each of which is done by a separate program.

1. Pre-process your source code to remove comments and do other tricks like expanding macros in C.
2. Check the syntax of your code to see if you have obeyed the rules of the language. If you have not, it will complain!
3. Convert the source code into assembly language-this is very close to machine code, but still understandable by humans. Allegedly.



4. Convert the assembly language into machine code-yep, we are talking bits and bytes, ones and zeros here.
5. Check that you have used things like functions and global variables in a consistent way. For example, if you have called a non-existent function, it will complain.
6. If you are trying to produce an executable from several source code files, work out how to fit them all together.
7. Work out how to produce something that the system's run-time loader will be able to load into memory and run.
8. Finally, write the executable on the filesystem.

The word *compiling* is often used to refer to just steps 1 to 4-the others are referred to as *linking*. Sometimes step 1 is referred to as *pre-processing* and steps 3-4 as *assembling*.

Fortunately, almost all this detail is hidden from you, as `cc` is a front end that manages calling all these programs with the right arguments for you; simply typing

```
% cc foobar.c
```



will cause **foobar.c** to be compiled by all the steps above. If you have more than one file to compile, just do something like

```
% cc foo.c bar.c
```



Note that the syntax checking is just that - checking the syntax. It will not check for any logical mistakes you may have made, like putting the program into an infinite loop, or using a bubble sort when you meant to use a binary sort.^[2]

There are lots and lots of options for `cc`, which are all in the manual page. Here are a few of the most important ones, with examples of how to use them.

-o filename

The output name of the file. If you do not use this option, `cc` will produce an executable called **a.out**.^[3]





```
% cc foobar.c          executable is a.out
% cc -o foobar foobar.c  executable is foobar
```

-c

Just compile the file, do not link it. Useful for toy programs where you just want to check the syntax, or if you are using a **Makefile**.



```
% cc -c foobar.c
```

This will produce an *object file* (not an executable) called **foobar.o**. This can be linked together with other object files into an executable.

-g

Create a debug version of the executable. This makes the compiler put information into the executable about which line of which source file corresponds to which function call. A debugger can use this information to show the source code as you step through the program, which is *very* useful; the disadvantage is that all this extra information makes the program much bigger. Normally, you compile with `-g` while you are developing a program and then compile a "release version" without `-g` when you are satisfied it works properly.



```
% cc -g foobar.c
```

This will produce a debug version of the program. ^[4]

-O

Create an optimized version of the executable. The compiler performs various clever tricks to try to produce an executable that runs faster than normal. You can add a number after the `-O` to specify a higher level of optimization, but this often exposes bugs in the compiler's optimizer.



```
% cc -O -o foobar foobar.c
```

This will produce an optimized version of **foobar**.

The following three flags will force `cc` to check that your code complies to the relevant international standard, often referred to as the ANSI standard, though strictly speaking it



is an ISO standard.

-Wall

Enable all the warnings which the authors of `cc` believe are worthwhile. Despite the name, it will not enable all the warnings `cc` is capable of.

-ansi

Turn off most, but not all, of the non-ANSI C features provided by `cc`. Despite the name, it does not guarantee strictly that your code will comply to the standard.

-pedantic

Turn off *all* `cc`'s non-ANSI C features.

Without these flags, `cc` will allow you to use some of its non-standard extensions to the standard. Some of these are very useful, but will not work with other compilers - in fact, one of the main aims of the standard is to allow people to write code that will work with any compiler on any system. This is known as *portable code*.

Generally, you should try to make your code as portable as possible, as otherwise you may have to completely rewrite the program later to get it to work somewhere else - and who knows what you may be using in a few years time?



```
% cc -Wall -ansi -pedantic -o foobar foobar.c
```

This will produce an executable **foobar** after checking **foobar.c** for standard compliance.

-lLibrary

Specify a function library to be used at link time.

The most common example of this is when compiling a program that uses some of the mathematical functions in C. Unlike most other platforms, these are in a separate library from the standard C one and you have to tell the compiler to add it.

The rule is that if the library is called **libsomething.a**, you give `cc` the argument `-lsomething`. For example, the math library is **libm.a**, so you give `cc` the argument `-lm`. A common "gotcha" with the math library is that it has to be the last library on the command line.



```
% cc -o foobar foobar.c -lm
```



This will link the math library functions into **foobar**.

If you are compiling C++ code, use `c++`. `c++` can also be invoked as `clang++` on FreeBSD.



```
% c++ -o foobar foobar.cc
```

This will both produce an executable **foobar** from the C++ source file **foobar.cc**.

2.4.1. Common cc Queries and Problems

2.4.1.1. I compiled a file called foobar.c and I cannot find an executable called foobar. Where has it gone?

Remember, `cc` will call the executable **a.out** unless you tell it differently. Use the `-o filename` option:



```
% cc -o foobar foobar.c
```

2.4.1.2. OK, I have an executable called foobar, I can see it when I run `ls`, but when I type in `foobar` at the command prompt it tells me there is no such file. Why can it not find it?

Unlike MS-DOS®, UNIX® does not look in the current directory when it is trying to find out which executable you want it to run, unless you tell it to. Type `./foobar`, which means "run the file called **foobar** in the current directory."

2.4.2. I called my executable test, but nothing happens when I run it. What is going on?

Most UNIX® systems have a program called `test` in **/usr/bin** and the shell is picking that one up before it gets to checking the current directory. Either type:



```
% ./test
```

or choose a better name for your program!

2.4.2.1. I compiled my program and it seemed to run all right at first, then there was an error and it said something about core

dumped. What does that mean?

The name *core dump* dates back to the very early days of UNIX®, when the machines used core memory for storing data. Basically, if the program failed under certain conditions, the system would write the contents of core memory to disk in a file called **core**, which the programmer could then pore over to find out what went wrong.

2.4.2.2. Fascinating stuff, but what I am supposed to do now?

Use a debugger to analyze the core (see [Debugging](#)).

2.4.2.3. When my program dumped core, it said something about a segmentation fault. What is that?

This basically means that your program tried to perform some sort of illegal operation on memory; UNIX® is designed to protect the operating system and other programs from rogue programs.

Common causes for this are:

- Trying to write to a NULL pointer, eg

```
char *foo = NULL;
strcpy(foo, "bang!");
```

- Using a pointer that has not been initialized, eg

```
char *foo;
strcpy(foo, "bang!");
```

The pointer will have some random value that, with luck, will point into an area of memory that is not available to your program and the kernel will kill your program before it can do any damage. If you are unlucky, it will point somewhere inside your own program and corrupt one of your data structures, causing the program to fail mysteriously.

- Trying to access past the end of an array, eg

```
int bar[20];
bar[27] = 6;
```

- Trying to store something in read-only memory, eg



```
char *foo = "My string";
strcpy(foo, "bang!");
```

UNIX® compilers often put string literals like "My string" into read-only areas of memory.

- Doing naughty things with `malloc()` and `free()`, eg

```
char bar[80];
free(bar);
```

or

```
char *foo = malloc(27);
free(foo);
free(foo);
```

Making one of these mistakes will not always lead to an error, but they are always bad practice. Some systems and compilers are more tolerant than others, which is why programs that ran well on one system can crash when you try them on an another.

2.4.2.4. Sometimes when I get a core dump it says bus error. It says in my UNIX® book that this means a hardware problem, but the computer still seems to be working. Is this true?

No, fortunately not (unless of course you really do have a hardware problem...). This is usually another way of saying that you accessed memory in a way you should not have.

2.4.2.5. This dumping core business sounds as though it could be quite useful, if I can make it happen when I want to. Can I do this, or do I have to wait until there is an error?

Yes, just go to another console or xterm, do

```
% ps
```



to find out the process ID of your program, and do

```
% kill -ABRT pid
```



where *pid* is the process ID you looked up.

This is useful if your program has got stuck in an infinite loop, for instance. If your program happens to trap SIGABRT, there are several other signals which have a similar effect.

Alternatively, you can create a core dump from inside your program, by calling the `abort()` function. See the manual page of [abort\(3\)](#) to learn more.

If you want to create a core dump from outside your program, but do not want the process to terminate, you can use the `gcore` program. See the manual page of [gcore\(1\)](#) for more information.

2.5. Make

2.5.1. What is make?

When you are working on a simple program with only one or two source files, typing in

```
% cc file1.c file2.c
```



is not too bad, but it quickly becomes very tedious when there are several files-and it can take a while to compile, too.

One way to get around this is to use object files and only recompile the source file if the source code has changed. So we could have something like:

```
% cc file1.o file2.o ... file37.c ...
```



if we had changed **file37.c**, but not any of the others, since the last time we compiled. This may speed up the compilation quite a bit, but does not solve the typing problem.

Or we could write a shell script to solve the typing problem, but it would have to re-compile everything, making it very inefficient on a large project.

What happens if we have hundreds of source files lying about? What if we are working in a team with other people who forget to tell us when they have changed one of their source files that we use?

Perhaps we could put the two solutions together and write something like a shell script that would contain some kind of magic rule saying when a source file needs compiling. Now all we need now is a program that can understand these rules, as it is a bit too complicated for the shell.



This program is called `make`. It reads in a file, called a *makefile*, that tells it how different files depend on each other, and works out which files need to be re-compiled and which ones do not. For example, a rule could say something like "if **fromboz.o** is older than **fromboz.c**, that means someone must have changed **fromboz.c**, so it needs to be re-compiled." The makefile also has rules telling `make` *how* to re-compile the source file, making it a much more powerful tool.

Makefiles are typically kept in the same directory as the source they apply to, and can be called **makefile**, **Makefile** or **MAKEFILE**. Most programmers use the name **Makefile**, as this puts it near the top of a directory listing, where it can easily be seen.^[5]

2.5.2. Example of Using `make`

Here is a very simple make file:

```
foo: foo.c
    cc -o foo foo.c
```

It consists of two lines, a dependency line and a creation line.

The dependency line here consists of the name of the program (known as the *target*), followed by a colon, then whitespace, then the name of the source file. When `make` reads this line, it looks to see if **foo** exists; if it exists, it compares the time **foo** was last modified to the time **foo.c** was last modified. If **foo** does not exist, or is older than **foo.c**, it then looks at the creation line to find out what to do. In other words, this is the rule for working out when **foo.c** needs to be re-compiled.

The creation line starts with a tab (press tab) and then the command you would type to create **foo** if you were doing it at a command prompt. If **foo** is out of date, or does not exist, `make` then executes this command to create it. In other words, this is the rule which tells `make` how to re-compile **foo.c**.

So, when you type `make`, it will make sure that **foo** is up to date with respect to your latest changes to **foo.c**. This principle can be extended to **Makefile**'s with hundreds of targets—in fact, on FreeBSD, it is possible to compile the entire operating system just by typing `make world` in the appropriate directory!

Another useful property of makefiles is that the targets do not have to be programs. For instance, we could have a make file that looks like this:

```
foo: foo.c
    cc -o foo foo.c
```



```
install:
    cp foo /home/me
```

We can tell make which target we want to make by typing:

```
% make target
```



`make` will then only look at that target and ignore any others. For example, if we type `make foo` with the makefile above, `make` will ignore the `install` target.

If we just type `make` on its own, `make` will always look at the first target and then stop without looking at any others. So if we typed `make` here, it will just go to the `foo` target, re-compile **foo** if necessary, and then stop without going on to the `install` target.

Notice that the `install` target does not actually depend on anything! This means that the command on the following line is always executed when we try to make that target by typing `make install`. In this case, it will copy **foo** into the user's home directory. This is often used by application makefiles, so that the application can be installed in the correct directory when it has been correctly compiled.

This is a slightly confusing subject to try to explain. If you do not quite understand how `make` works, the best thing to do is to write a simple program like "hello world" and a make file like the one above and experiment. Then progress to using more than one source file, or having the source file include a header file. `touch` is very useful here-it changes the date on a file without you having to edit it.

2.5.3. Make and include-files

C code often starts with a list of files to include, for example `stdio.h`. Some of these files are system-include files, some of them are from the project you are now working on:

```
#include <stdio.h>
#include "foo.h"

int main(...
```

To make sure that this file is recompiled the moment **foo.h** is changed, you have to add it in your **Makefile**:

```
foo: foo.c foo.h
```



The moment your project is getting bigger and you have more and more own include-files to maintain, it will be a pain to keep track of all include files and the files which are depending on it. If you change an include-file but forget to recompile all the files which are depending on it, the results will be devastating. `clang` has an option to analyze your files and to produce a list of include-files and their dependencies: `-MM`.

If you add this to your Makefile:

```
depend:
    cc -E -MM *.c > .depend
```

and run `make depend`, the file **.depend** will appear with a list of object-files, C-files and the include-files:

```
foo.o: foo.c foo.h
```

If you change **foo.h**, next time you run `make` all files depending on **foo.h** will be recompiled.

Do not forget to run `make depend` each time you add an include-file to one of your files.

2.5.4. FreeBSD Makefiles

Makefiles can be rather complicated to write. Fortunately, BSD-based systems like FreeBSD come with some very powerful ones as part of the system. One very good example of this is the FreeBSD ports system. Here is the essential part of a typical ports **Makefile**:

```
MASTER_SITES=    ftp://freefall.cdrom.com/pub/FreeBSD/LOCAL_PORTS/
DISTFILES=        scheme-microcode+dist-7.3-freebsd.tgz

.include <bsd.port.mk>
```

Now, if we go to the directory for this port and type `make`, the following happens:

1. A check is made to see if the source code for this port is already on the system.
2. If it is not, an FTP connection to the URL in `MASTER_SITES` is set up to download the source.



3. The checksum for the source is calculated and compared it with one for a known, good, copy of the source. This is to make sure that the source was not corrupted while in transit.
4. Any changes required to make the source work on FreeBSD are applied-this is known as *patching*.
5. Any special configuration needed for the source is done. (Many UNIX® program distributions try to work out which version of UNIX® they are being compiled on and which optional UNIX® features are present-this is where they are given the information in the FreeBSD ports scenario).
6. The source code for the program is compiled. In effect, we change to the directory where the source was unpacked and do `make`-the program's own make file has the necessary information to build the program.
7. We now have a compiled version of the program. If we wish, we can test it now; when we feel confident about the program, we can type `make install`. This will cause the program and any supporting files it needs to be copied into the correct location; an entry is also made into a `package` database, so that the port can easily be uninstalled later if we change our mind about it.

Now I think you will agree that is rather impressive for a four line script!

The secret lies in the last line, which tells `make` to look in the system makefile called **bsd.port.mk**. It is easy to overlook this line, but this is where all the clever stuff comes from-someone has written a makefile that tells `make` to do all the things above (plus a couple of other things I did not mention, including handling any errors that may occur) and anyone can get access to that just by putting a single line in their own make file!

If you want to have a look at these system makefiles, they are in **/usr/share/mk**, but it is probably best to wait until you have had a bit of practice with makefiles, as they are very complicated (and if you do look at them, make sure you have a flask of strong coffee handy!)

2.5.5. More Advanced Uses of `make`

`Make` is a very powerful tool, and can do much more than the simple example above shows. Unfortunately, there are several different versions of `make`, and they all differ considerably. The best way to learn what they can do is probably to read the documentation-hopefully this introduction will have given you a base from which you can do this. The [make\(1\)](#) manual page offers a comprehensive discussion of variables, arguments, and how to use `make`.



Many applications in the ports use GNU make, which has a very good set of "info" pages. If you have installed any of these ports, GNU make will automatically have been installed as `gmake`. It is also available as a port and package in its own right.

To view the info pages for GNU make, you will have to edit **dir** in the **/usr/local/info** directory to add an entry for it. This involves adding a line like

```
* Make: (make).                The GNU Make utility.
```

to the file. Once you have done this, you can type `info` and then select `make` from the menu (or in Emacs, do `C-h i`).

2.6. Debugging

2.6.1. Introduction to Available Debuggers

Using a debugger allows running the program under more controlled circumstances. Typically, it is possible to step through the program a line at a time, inspect the value of variables, change them, tell the debugger to run up to a certain point and then stop, and so on. It is also possible to attach to a program that is already running, or load a core file to investigate why the program crashed.

This section is intended to be a quick introduction to using debuggers and does not cover specialized topics such as debugging the kernel. For more information about that, refer to [Kernel Debugging](#).

The standard debugger supplied with FreeBSD is called `lldb` (LLVM debugger). As it is part of the standard installation for that release, there is no need to do anything special to use it. It has good command help, accessible via the `help` command, as well as [a web tutorial and documentation](#).

Note

The `lldb` command is also available [from ports or packages](#) as `devel/llvm`.

The other debugger available with FreeBSD is called `gdb` (GNU debugger). Unlike `lldb`, it is not installed by default on FreeBSD; to use it, [install `devel/gdb`](#) from ports or packages. It has excellent on-line help, as well as a set of info pages.

The two debuggers have a similar feature set, so which one to use is largely a matter of taste. If familiar with one only, use that one. People familiar with neither or both but wanting to use one from inside Emacs will need to use `gdb` as `lldb` is unsupported by Emacs. Otherwise, try both and see which one you prefer.



2.6.2. Using lldb

2.6.2.1. Starting lldb

Start up lldb by typing

```
% lldb -- progname
```



2.6.2.2. Running a Program with lldb

Compile the program with `-g` to get the most out of using `lldb`. It will work without, but will only display the name of the function currently running, instead of the source code. If it displays a line like:

```
Breakpoint 1: where = temp`main, address = ...
```



(without an indication of source code filename and line number) when setting a breakpoint, this means that the program was not compiled with `-g`.

Tip

Most `lldb` commands have shorter forms that can be used instead. The longer forms are used here for clarity.

At the `lldb` prompt, type `breakpoint set -n main`. This will tell the debugger not to display the preliminary set-up code in the program being run and to stop execution at the beginning of the program's code. Now type `process launch` to actually start the program- it will start at the beginning of the set-up code and then get stopped by the debugger when it calls `main()`.

To step through the program a line at a time, type `thread step-over`. When the program gets to a function call, step into it by typing `thread step-in`. Once in a function call, return from it by typing `thread step-out` or use `up` and `down` to take a quick look at the caller.

Here is a simple example of how to spot a mistake in a program with `lldb`. This is our program (with a deliberate mistake):



```
#include <stdio.h>

int bazz(int anint);

main() {
    int i;

    printf("This is my program\n");
    bazz(i);
    return 0;
}

int bazz(int anint) {
    printf("You gave me %d\n", anint);
    return anint;
}
```

This program sets `i` to be 5 and passes it to a function `bazz()` which prints out the number we gave it.

Compiling and running the program displays

```
% cc -g -o temp temp.c
% ./temp
This is my program
anint = -5360
```



That is not what was expected! Time to see what is going on!

```
% lldb -- temp
(lldb) target create "temp"
Current executable set to 'temp' (x86_64).
(lldb) breakpoint set -n main                               Skip the set-up code
Breakpoint 1: where = temp`main + 15 at temp.c:8:2, address = 0x0000000002012ef          lldb puts
(lldb) process launch                                       Run as far as main()
Process 9992 launching
Process 9992 launched: '/home/pauamma/tmp/temp' (x86_64)   Program starts running

Process 9992 stopped
* thread #1, name = 'temp', stop reason = breakpoint 1.1    lldb stops at main()
    frame #0: 0x0000000002012ef temp`main at temp.c:8:2
    5   main() {
    6       int i;
```



```

7
-> 8      printf("This is my program\n");
9      bazz(i);
10     return 0;
11 }
(lldb) thread step-over          Go to next line
This is my program              Program prints out
Process 9992 stopped

* thread #1, name = 'temp', stop reason = step over
  frame #0: 0x000000000201300 temp`main at temp.c:9:7
    6      int i;
    7
    8      printf("This is my program\n");
-> 9      bazz(i);
10     return 0;
11 }
12
(lldb) thread step-in           step into bazz()
Process 9992 stopped

* thread #1, name = 'temp', stop reason = step in
  frame #0: 0x00000000020132b temp`bazz(anint=-5360) at temp.c:14:29 lldb displays stack frame
11 }
12
13 int bazz(int anint) {
-> 14     printf("You gave me %d\n", anint);
15     return anint;
16 }
(lldb)

```

Hang on a minute! How did anint get to be -5360? Was it not set to 5 in main()? Let us move up to main() and have a look.

```

(lldb) up                      Move up call stack
frame #1: 0x00000000020130b temp`main at temp.c:9:2          lldb displays stack frame
    6      int i;
    7
    8      printf("This is my program\n");
-> 9      bazz(i);
10     return 0;
11 }
12
(lldb) frame variable i        Show us the value of i
(int) i = -5360                lldb displays -5360

```



Oh dear! Looking at the code, we forgot to initialize `i`. We meant to put

```
...
main() {
    int i;

    i = 5;
    printf("This is my program\n");
    ...
}
```

but we left the `i=5;` line out. As we did not initialize `i`, it had whatever number happened to be in that area of memory when the program ran, which in this case happened to be `-5360`.

Note

The `lldb` command displays the stack frame every time we go into or out of a function, even if we are using `up` and `down` to move around the call stack. This shows the name of the function and the values of its arguments, which helps us keep track of where we are and what is going on. (The stack is a storage area where the program stores information about the arguments passed to functions and where to go when it returns from a function call.)

2.6.2.3. Examining a Core File with lldb

A core file is basically a file which contains the complete state of the process when it crashed. In "the good old days", programmers had to print out hex listings of core files and sweat over machine code manuals, but now life is a bit easier. Incidentally, under FreeBSD and other 4.4BSD systems, a core file is called **progrname.core** instead of just **core**, to make it clearer which program a core file belongs to.

To examine a core file, specify the name of the core file in addition to the program itself. Instead of starting up `lldb` in the usual way, type `lldb -c progrname.core -- progrname`.

The debugger will display something like this:

```
% lldb -c progrname.core -- progrname
(lldb) target create "progrname" --core "progrname.core"
Core file '/home/pauamma/tmp/progrname.core' (x86_64) was loaded.
(lldb)
```



In this case, the program was called **progrname**, so the core file is called **progrname.core**. The debugger does not display why the program crashed or where. For this, use `thread backtrace all`. This will also show how the function where the program dumped core was called.



```
(lldb) thread backtrace all
thread #1, name = 'progrname', stop reason = signal SIGSEGV
  frame #0: 0x0000000000201347 progrname`bazz(anint=5) at temp2.c:17:10
  frame #1: 0x0000000000201312 progrname`main at temp2.c:10:2
  frame #2: 0x000000000020110f progrname`_start(ap=<unavailable>, cleanup=<unavailable>) at crt1
(lldb)
```

SIGSEGV indicates that the program tried to access memory (run code or read/write data usually) at a location that does not belong to it, but does not give any specifics. For that, look at the source code at line 10 of file temp2.c, in `bazz()`. The backtrace also says that in this case, `bazz()` was called from `main()`.

2.6.2.4. Attaching to a Running Program with lldb

One of the neatest features about `lldb` is that it can attach to a program that is already running. Of course, that requires sufficient permissions to do so. A common problem is stepping through a program that forks and wanting to trace the child, but the debugger will only trace the parent.

To do that, start up another `lldb`, use `ps` to find the process ID for the child, and do



```
(lldb) process attach -p pid
```

in `lldb`, and then debug as usual.

For that to work well, the code that calls `fork` to create the child needs to do something like the following (courtesy of the `gdb` info pages):

```
...
if ((pid = fork()) < 0)          /* _Always_ check this */
    error();
else if (pid == 0) {           /* child */
    int PauseMode = 1;

    while (PauseMode)
        sleep(10);            /* Wait until someone attaches to us */
    ...
```



```
} else {
    ...
    /* parent */
```

Now all that is needed is to attach to the child, set `PauseMode` to `0` with `expr` `PauseMode = 0` and wait for the `sleep()` call to return.

2.6.3. Remote Debugging Using LLDB

Note

The described functionality is available starting with LLDB version 12.0.0. Users of FreeBSD releases containing an earlier LLDB version may wish to use the snapshot available in [ports or packages](#), as [devel/llvm-devel](#).

Starting with LLDB 12.0.0, remote debugging is supported on FreeBSD. This means that `lldb-server` can be started to debug a program on one host, while the interactive `lldb` client connects to it from another one.

To launch a new process to be debugged remotely, run `lldb-server` on the remote server by typing

```
% lldb-server g host:port -- progname
```



The process will be stopped immediately after launching, and `lldb-server` will wait for the client to connect.

Start `lldb` locally and type the following command to connect to the remote server:

```
(lldb) gdb-remote host:port
```



`lldb-server` can also attach to a running process. To do that, type the following on the remote server:

```
% lldb-server g host:port --attach pid-or-name
```



2.6.4. Using gdb

2.6.4.1. Starting gdb



Start up gdb by typing

```
% gdb progname
```



although many people prefer to run it inside Emacs. To do this, type:

```
M-x gdb RET progname RET
```



Finally, for those finding its text-based command-prompt style off-putting, there is a graphical front-end for it ([devel/xxgdb](#)) in the Ports Collection.

2.6.4.2. Running a Program with gdb

Compile the program with `-g` to get the most out of using `gdb`. It will work without, but will only display the name of the function currently running, instead of the source code. A line like:

```
... (no debugging symbols found) ...
```



when `gdb` starts up means that the program was not compiled with `-g`.

At the `gdb` prompt, type `break main`. This will tell the debugger to skip the preliminary set-up code in the program being run and to stop execution at the beginning of the program's code. Now type `run` to start the program- it will start at the beginning of the set-up code and then get stopped by the debugger when it calls `main()`.

To step through the program a line at a time, press `n`. When at a function call, step into it by pressing `s`. Once in a function call, return from it by pressing `f`, or use `up` and `down` to take a quick look at the caller.

Here is a simple example of how to spot a mistake in a program with `gdb`. This is our program (with a deliberate mistake):

```
#include <stdio.h>

int bazz(int anint);

main() {
    int i;

    printf("This is my program\n");
```




```

        bazz(i);
        return 0;
    }

    int bazz(int anint) {
        printf("You gave me %d\n", anint);
        return anint;
    }

```

This program sets `i` to be 5 and passes it to a function `bazz()` which prints out the number we gave it.

Compiling and running the program displays

```

% cc -g -o temp temp.c
% ./temp
This is my program
anint = 4231

```



That was not what we expected! Time to see what is going on!

```

% gdb temp
GDB is free software and you are welcome to distribute copies of it
under certain conditions; type "show copying" to see the conditions.
There is absolutely no warranty for GDB; type "show warranty" for details.
GDB 4.13 (i386-unknown-freebsd), Copyright 1994 Free Software Foundation, Inc.
(gdb) break main                               Skip the set-up code
Breakpoint 1 at 0x160f: file temp.c, line 9.     gdb puts breakpoint at main()
(gdb) run                                       Run as far as main()
Starting program: /home/james/tmp/temp          Program starts running

Breakpoint 1, main () at temp.c:9              gdb stops at main()
(gdb) n                                       Go to next line
This is my program                             Program prints out
(gdb) s                                       step into bazz()
bazz (anint=4231) at temp.c:17                 gdb displays stack frame
(gdb)

```



Hang on a minute! How did `anint` get to be 4231? Was it not set to 5 in `main()`? Let us move up to `main()` and have a look.





```
(gdb) up
#1  0x1625 in main () at temp.c:11
(gdb) p i
$1 = 4231
```

Move up call stack
gdb displays stack frame
Show us the value of i
gdb displays 4231

Oh dear! Looking at the code, we forgot to initialize i. We meant to put

```
...
main() {
    int i;

    i = 5;
    printf("This is my program\n");
    ...
}
```

but we left the `i=5;` line out. As we did not initialize i, it had whatever number happened to be in that area of memory when the program ran, which in this case happened to be 4231.

Note

The `gdb` command displays the stack frame every time we go into or out of a function, even if we are using `up` and `down` to move around the call stack. This shows the name of the function and the values of its arguments, which helps us keep track of where we are and what is going on. (The stack is a storage area where the program stores information about the arguments passed to functions and where to go when it returns from a function call.)

2.6.4.3. Examining a Core File with `gdb`

A core file is basically a file which contains the complete state of the process when it crashed. In "the good old days", programmers had to print out hex listings of core files and sweat over machine code manuals, but now life is a bit easier. Incidentally, under FreeBSD and other 4.4BSD systems, a core file is called **progname.core** instead of just **core**, to make it clearer which program a core file belongs to.

To examine a core file, start up `gdb` in the usual way. Instead of typing `break` or `run`, type

```
(gdb) core progname.core
```



If the core file is not in the current directory, type `dir /path/to/core/file` first.

The debugger should display something like this:



```
% gdb progname
GDB is free software and you are welcome to distribute copies of it
under certain conditions; type "show copying" to see the conditions.
There is absolutely no warranty for GDB; type "show warranty" for details.
GDB 4.13 (i386-unknown-freebsd), Copyright 1994 Free Software Foundation, Inc.
(gdb) core progname.core
Core was generated by `progname'.
Program terminated with signal 11, Segmentation fault.
Cannot access memory at address 0x7020796d.
#0  0x164a in bazz (anint=0x5) at temp.c:17
(gdb)
```

In this case, the program was called **progname**, so the core file is called **progname.core**. We can see that the program crashed due to trying to access an area in memory that was not available to it in a function called `bazz`.

Sometimes it is useful to be able to see how a function was called, as the problem could have occurred a long way up the call stack in a complex program. `bt` causes `gdb` to print out a back-trace of the call stack:



```
(gdb) bt
#0  0x164a in bazz (anint=0x5) at temp.c:17
#1  0xefbfd888 in end ()
#2  0x162c in main () at temp.c:11
(gdb)
```

The `end()` function is called when a program crashes; in this case, the `bazz()` function was called from `main()`.

2.6.4.4. Attaching to a Running Program with `gdb`

One of the neatest features about `gdb` is that it can attach to a program that is already running. Of course, that requires sufficient permissions to do so. A common problem is stepping through a program that forks and wanting to trace the child, but the debugger will only trace the parent.

To do that, start up another `gdb`, use `ps` to find the process ID for the child, and do





```
(gdb) attach pid
```

in `gdb`, and then debug as usual.

For that to work well, the code that calls `fork` to create the child needs to do something like the following (courtesy of the `gdb` info pages):

```
...
if ((pid = fork()) < 0)          /* _Always_ check this */
    error();
else if (pid == 0) {            /* child */
    int PauseMode = 1;

    while (PauseMode)
        sleep(10);            /* Wait until someone attaches to us */
    ...
} else {                        /* parent */
    ...
```

Now all that is needed is to attach to the child, set `PauseMode` to `0`, and wait for the `sleep()` call to return!

2.7. Using Emacs as a Development Environment

2.7.1. Emacs

Emacs is a highly customizable editor—indeed, it has been customized to the point where it is more like an operating system than an editor! Many developers and sysadmins do in fact spend practically all their time working inside Emacs, leaving it only to log out.

It is impossible even to summarize everything Emacs can do here, but here are some of the features of interest to developers:

- Very powerful editor, allowing search-and-replace on both strings and regular expressions (patterns), jumping to start/end of block expression, etc, etc.
- Pull-down menus and online help.
- Language-dependent syntax highlighting and indentation.
- Completely customizable.
- You can compile and debug programs within Emacs.



- On a compilation error, you can jump to the offending line of source code.
- Friendly-ish front-end to the `info` program used for reading GNU hypertext documentation, including the documentation on Emacs itself.
- Friendly front-end to `gdb`, allowing you to look at the source code as you step through your program.

And doubtless many more that have been overlooked.

Emacs can be installed on FreeBSD using the [editors/emacs](#) port.

Once it is installed, start it up and do `C-h t` to read an Emacs tutorial-that means hold down `control`, press `h`, let go of `control`, and then press `t`. (Alternatively, you can use the mouse to select Emacs Tutorial from the **Help** menu.)

Although Emacs does have menus, it is well worth learning the key bindings, as it is much quicker when you are editing something to press a couple of keys than to try to find the mouse and then click on the right place. And, when you are talking to seasoned Emacs users, you will find they often casually throw around expressions like “M-x replace-s RET foo RET bar RET” so it is useful to know what they mean. And in any case, Emacs has far too many useful functions for them to all fit on the menu bars.

Fortunately, it is quite easy to pick up the key-bindings, as they are displayed next to the menu item. My advice is to use the menu item for, say, opening a file until you understand how it works and feel confident with it, then try doing `C-x C-f`. When you are happy with that, move on to another menu command.

If you cannot remember what a particular combination of keys does, select Describe Key from the **Help** menu and type it in-Emacs will tell you what it does. You can also use the Command Apropos menu item to find out all the commands which contain a particular word in them, with the key binding next to it.

By the way, the expression above means hold down the `Meta` key, press `x`, release the `Meta` key, type `replace-s` (short for `replace-string`-another feature of Emacs is that you can abbreviate commands), press the `return` key, type `foo` (the string you want replaced), press the `return` key, type `bar` (the string you want to replace `foo` with) and press `return` again. Emacs will then do the search-and-replace operation you have just requested.

If you are wondering what on earth `Meta` is, it is a special key that many UNIX® workstations have. Unfortunately, PC's do not have one, so it is usually `alt` (or if you are unlucky, the `escape` key).

Oh, and to get out of Emacs, do `C-x C-c` (that means hold down the `control` key, press `x`, press `c` and release the `control` key). If you have any unsaved files open, Emacs will 

ask you if you want to save them. (Ignore the bit in the documentation where it says `C-z` is the usual way to leave Emacs—that leaves Emacs hanging around in the background, and is only really useful if you are on a system which does not have virtual terminals).

2.7.2. Configuring Emacs

Emacs does many wonderful things; some of them are built in, some of them need to be configured.

Instead of using a proprietary macro language for configuration, Emacs uses a version of Lisp specially adapted for editors, known as Emacs Lisp. Working with Emacs Lisp can be quite helpful if you want to go on and learn something like Common Lisp. Emacs Lisp has many features of Common Lisp, although it is considerably smaller (and thus easier to master).

The best way to learn Emacs Lisp is to download the [Emacs Tutorial](#)

However, there is no need to actually know any Lisp to get started with configuring Emacs, as I have included a sample **.emacs**, which should be enough to get you started. Just copy it into your home directory and restart Emacs if it is already running; it will read the commands from the file and (hopefully) give you a useful basic setup.

2.7.3. A Sample .emacs

Unfortunately, there is far too much here to explain it in detail; however there are one or two points worth mentioning.

- Everything beginning with a `;` is a comment and is ignored by Emacs.
- In the first line, the `-- Emacs-Lisp --` is so that we can edit **.emacs** itself within Emacs and get all the fancy features for editing Emacs Lisp. Emacs usually tries to guess this based on the filename, and may not get it right for **.emacs**.
- The `tab` key is bound to an indentation function in some modes, so when you press the tab key, it will indent the current line of code. If you want to put a tab character in whatever you are writing, hold the `control` key down while you are pressing the `tab` key.
- This file supports syntax highlighting for C, C++, Perl, Lisp and Scheme, by guessing the language from the filename.
- Emacs already has a pre-defined function called `next-error`. In a compilation output window, this allows you to move from one compilation error to the next by doing `M-n`; we define a complementary function, `previous-error`, that allows you to go to a previous error by doing `M-p`. The nicest feature of all is that `C-c C-c` will



open up the source file in which the error occurred and jump to the appropriate line.

- We enable Emacs's ability to act as a server, so that if you are doing something outside Emacs and you want to edit a file, you can just type in



```
% emacsclient filename
```

and then you can edit the file in your Emacs!^[6]

Example 1. A Sample .emacs

```
;; -*-Emacs-Lisp-*-

;; This file is designed to be re-evald; use the variable first-time
;; to avoid any problems with this.
(defvar first-time t
  "Flag signifying this is the first time that .emacs has been evaled")

;; Meta
(global-set-key "\M- " 'set-mark-command)
(global-set-key "\M-\C-h" 'backward-kill-word)
(global-set-key "\M-\C-r" 'query-replace)
(global-set-key "\M-r" 'replace-string)
(global-set-key "\M-g" 'goto-line)
(global-set-key "\M-h" 'help-command)

;; Function keys
(global-set-key [f1] 'manual-entry)
(global-set-key [f2] 'info)
(global-set-key [f3] 'repeat-complex-command)
(global-set-key [f4] 'advertised-undo)
(global-set-key [f5] 'eval-current-buffer)
(global-set-key [f6] 'buffer-menu)
(global-set-key [f7] 'other-window)
(global-set-key [f8] 'find-file)
(global-set-key [f9] 'save-buffer)
(global-set-key [f10] 'next-error)
(global-set-key [f11] 'compile)
(global-set-key [f12] 'grep)
(global-set-key [C-f1] 'compile)
(global-set-key [C-f2] 'grep)
(global-set-key [C-f3] 'next-error)
(global-set-key [C-f4] 'previous-error)
```



```
(global-set-key [C-f5] 'display-faces)
(global-set-key [C-f8] 'dired)
(global-set-key [C-f10] 'kill-compilation)

;; Keypad bindings
(global-set-key [up] "\C-p")
(global-set-key [down] "\C-n")
(global-set-key [left] "\C-b")
(global-set-key [right] "\C-f")
(global-set-key [home] "\C-a")
(global-set-key [end] "\C-e")
(global-set-key [prior] "\M-v")
(global-set-key [next] "\C-v")
(global-set-key [C-up] "\M-\C-b")
(global-set-key [C-down] "\M-\C-f")
(global-set-key [C-left] "\M-b")
(global-set-key [C-right] "\M-f")
(global-set-key [C-home] "\M-<")
(global-set-key [C-end] "\M->")
(global-set-key [C-prior] "\M-<")
(global-set-key [C-next] "\M->")

;; Mouse
(global-set-key [mouse-3] 'imenu)

;; Misc
(global-set-key [C-tab] "\C-q\t") ; Control tab quotes a tab.
(setq backup-by-copying-when-mismatch t)

;; Treat 'y' or <CR> as yes, 'n' as no.
(fset 'yes-or-no-p 'y-or-n-p)
(define-key query-replace-map [return] 'act)
(define-key query-replace-map [?\C-m] 'act)

;; Load packages
(require 'desktop)
(require 'tar-mode)

;; Pretty diff mode
(autoload 'ediff-buffers "ediff" "Intelligent Emacs interface to diff"
(autoload 'ediff-files "ediff" "Intelligent Emacs interface to diff" t)
(autoload 'ediff-files-remote "ediff"
  "Intelligent Emacs interface to diff")

(if first-time
  (setq auto-mode-alist
```




```

      (append '("\.cpp$" . c++-mode)
              ("\.hpp$" . c++-mode)
              ("\.lsp$" . lisp-mode)
              ("\.scm$" . scheme-mode)
              ("\.pl$" . perl-mode)
              ) auto-mode-alist)))

;; Auto font lock mode
(defvar font-lock-auto-mode-list
  (list 'c-mode 'c++-mode 'c++-c-mode 'emacs-lisp-mode 'lisp-mode 'perl-mode
        "List of modes to always start in font-lock-mode")

  (defvar font-lock-mode-keyword-alist
    '((c++-c-mode . c-font-lock-keywords)
      (perl-mode . perl-font-lock-keywords))
    "Associations between modes and keywords")

  (defun font-lock-auto-mode-select ()
    "Automatically select font-lock-mode if the current major mode is in
    (if (memq major-mode font-lock-auto-mode-list)
        (progn
          (font-lock-mode t))
        )
    )

  (global-set-key [M-f1] 'font-lock-fontify-buffer)

;; New dabbrev stuff
;(require 'new-dabbrev)
(setq dabbrev-always-check-other-buffers t)
(setq dabbrev-abbrev-char-regexp "\sw\\|\\s_")
(add-hook 'emacs-lisp-mode-hook
  '(lambda ()
    (set (make-local-variable 'dabbrev-case-fold-search) nil)
    (set (make-local-variable 'dabbrev-case-replace) nil)))
(add-hook 'c-mode-hook
  '(lambda ()
    (set (make-local-variable 'dabbrev-case-fold-search) nil)
    (set (make-local-variable 'dabbrev-case-replace) nil)))
(add-hook 'text-mode-hook
  '(lambda ()
    (set (make-local-variable 'dabbrev-case-fold-search) t)
    (set (make-local-variable 'dabbrev-case-replace) t)))

;; C++ and C mode...
(defun my-c++-mode-hook ()

```



```
(setq tab-width 4)
(define-key c++-mode-map "\C-m" 'reindent-then-newline-and-indent)
(define-key c++-mode-map "\C-ce" 'c-comment-edit)
(setq c++-auto-hungry-initial-state 'none)
(setq c++-delete-function 'backward-delete-char)
(setq c++-tab-always-indent t)
(setq c-indent-level 4)
(setq c-continued-statement-offset 4)
(setq c++-empty-arglist-indent 4))

(defun my-c-mode-hook ()
  (setq tab-width 4)
  (define-key c-mode-map "\C-m" 'reindent-then-newline-and-indent)
  (define-key c-mode-map "\C-ce" 'c-comment-edit)
  (setq c-auto-hungry-initial-state 'none)
  (setq c-delete-function 'backward-delete-char)
  (setq c-tab-always-indent t)
;; BSD-ish indentation style
  (setq c-indent-level 4)
  (setq c-continued-statement-offset 4)
  (setq c-brace-offset -4)
  (setq c-argdecl-indent 0)
  (setq c-label-offset -4))

;; Perl mode
(defun my-perl-mode-hook ()
  (setq tab-width 4)
  (define-key c++-mode-map "\C-m" 'reindent-then-newline-and-indent)
  (setq perl-indent-level 4)
  (setq perl-continued-statement-offset 4))

;; Scheme mode...
(defun my-scheme-mode-hook ()
  (define-key scheme-mode-map "\C-m" 'reindent-then-newline-and-indent)

;; Emacs-Lisp mode...
(defun my-lisp-mode-hook ()
  (define-key lisp-mode-map "\C-m" 'reindent-then-newline-and-indent)
  (define-key lisp-mode-map "\C-i" 'lisp-indent-line)
  (define-key lisp-mode-map "\C-j" 'eval-print-last-sexp))

;; Add all of the hooks...
(add-hook 'c++-mode-hook 'my-c++-mode-hook)
(add-hook 'c-mode-hook 'my-c-mode-hook)
(add-hook 'scheme-mode-hook 'my-scheme-mode-hook)
(add-hook 'emacs-lisp-mode-hook 'my-lisp-mode-hook)
```



```
(add-hook 'lisp-mode-hook 'my-lisp-mode-hook)
(add-hook 'perl-mode-hook 'my-perl-mode-hook)

;; Complement to next-error
(defun previous-error (n)
  "Visit previous compilation error message and corresponding source code"
  (interactive "p")
  (next-error (- n)))

;; Misc...
(transient-mark-mode 1)
(setq mark-even-if-inactive t)
(setq visible-bell nil)
(setq next-line-add-newlines nil)
(setq compile-command "make")
(setq suggest-key-bindings nil)
(put 'eval-expression 'disabled nil)
(put 'narrow-to-region 'disabled nil)
(put 'set-goal-column 'disabled nil)
(if (>= emacs-major-version 21)
    (setq show-trailing-whitespace t))

;; Elisp archive searching
(autoload 'format-lisp-code-directory "lispdir" nil t)
(autoload 'lisp-dir-apropos "lispdir" nil t)
(autoload 'lisp-dir-retrieve "lispdir" nil t)
(autoload 'lisp-dir-verify "lispdir" nil t)

;; Font lock mode
(defun my-make-face (face color &optional bold)
  "Create a face from a color and optionally make it bold"
  (make-face face)
  (copy-face 'default face)
  (set-face-foreground face color)
  (if bold (make-face-bold face))
  )

(if (eq window-system 'x)
    (progn
      (my-make-face 'blue "blue")
      (my-make-face 'red "red")
      (my-make-face 'green "dark green")
      (setq font-lock-comment-face 'blue)
      (setq font-lock-string-face 'bold)
      (setq font-lock-type-face 'bold)
      (setq font-lock-keyword-face 'bold)
```



```

(setq font-lock-function-name-face 'red)
(setq font-lock-doc-string-face 'green)
(add-hook 'find-file-hooks 'font-lock-auto-mode-select)

(setq baud-rate 1000000)
(global-set-key "\C-cmm" 'menu-bar-mode)
(global-set-key "\C-cms" 'scroll-bar-mode)
(global-set-key [backspace] 'backward-delete-char)
; (global-set-key [delete

(standard-display-european t)
(load-library "iso-transl"))

;; X11 or PC using direct screen writes
(if window-system
    (progn
      ;; (global-set-key [M-f1] 'hilit-repaint-command)
      ;; (global-set-key [M-f2] [?\C-u M-f1])
      (setq hilit-mode-enable-list
        '(not text-mode c-mode c++-mode emacs-lisp-mode lisp-mode
          scheme-mode)
        hilit-auto-highlight nil
        hilit-auto-rehighlight 'visible
        hilit-inhibit-hooks nil
        hilit-inhibit-rebinding t)
      (require 'hilit19)
      (require 'paren))
      (setq baud-rate 2400) ; For slow serial connections
    )

    ;; TTY type terminal
    (if (and (not window-system)
              (not (equal system-type 'ms-dos)))
        (progn
          (if first-time
              (progn
                (keyboard-translate ?\C-h ?\C-?)
                (keyboard-translate ?\C-? ?\C-h))))))

    ;; Under UNIX
    (if (not (equal system-type 'ms-dos))
        (progn
          (if first-time
              (server-start))))

    ;; Add any face changes here
    (add-hook 'term-setup-hook 'my-term-setup-hook)

```



```
(defun my-term-setup-hook ()
  (if (eq window-system 'pc)
      (progn
        ;; (set-face-background 'default "red")
      )))

;; Restore the "desktop" - do this as late as possible
(if first-time
    (progn
      (desktop-load-default)
      (desktop-read)))

;; Indicate that this file has been read at least once
(setq first-time nil)

;; No need to debug anything now

(setq debug-on-error nil)

;; All done
(message "All done, %s%s" (user-login-name) ".")
```

2.7.4. Extending the Range of Languages Emacs Understands

Now, this is all very well if you only want to program in the languages already catered for in **.emacs** (C, C++, Perl, Lisp and Scheme), but what happens if a new language called "whizbang" comes out, full of exciting features?

The first thing to do is find out if whizbang comes with any files that tell Emacs about the language. These usually end in **.el**, short for "Emacs Lisp". For example, if whizbang is a FreeBSD port, we can locate these files by doing

```
% find /usr/ports/lang/whizbang -name "*.el" -print
```



and install them by copying them into the Emacs site Lisp directory. On FreeBSD, this is **/usr/local/share/emacs/site-lisp**.

So for example, if the output from the find command was





```
/usr/ports/lang/whizbang/work/misc/whizbang.el
```

we would do



```
# cp /usr/ports/lang/whizbang/work/misc/whizbang.el /usr/local/share/emacs/site-lisp
```

Next, we need to decide what extension whizbang source files have. Let us say for the sake of argument that they all end in **.wiz**. We need to add an entry to our **.emacs** to make sure Emacs will be able to use the information in **whizbang.el**.

Find the auto-mode-alist entry in **.emacs** and add a line for whizbang, such as:

```
...
("\\.lsp$" . lisp-mode)
("\\.wiz$" . whizbang-mode)
("\\.scm$" . scheme-mode)
...
```

This means that Emacs will automatically go into whizbang-mode when you edit a file ending in **.wiz**.

Just below this, you will find the font-lock-auto-mode-list entry. Add whizbang-mode to it like so:

```
; Auto font lock mode
(defvar font-lock-auto-mode-list
  (list 'c-mode 'c++-mode 'c++-c-mode 'emacs-lisp-mode 'whizbang-mode 'lisp
    "List of modes to always start in font-lock-mode"))
```

This means that Emacs will always enable font-lock-mode (ie syntax highlighting) when editing a **.wiz** file.

And that is all that is needed. If there is anything else you want done automatically when you open up **.wiz**, you can add a whizbang-mode hook (see my-scheme-mode-hook for a simple example that adds auto-indent).

2.8. Further Reading

For information about setting up a development environment for contributing fixes to FreeBSD itself, please see [development\(7\)](#).



- Brian Harvey and Matthew Wright *Simply Scheme* MIT 1994. ISBN 0-262-08226-8
- Randall Schwartz *Learning Perl* O'Reilly 1993 ISBN 1-56592-042-2
- Patrick Henry Winston and Berthold Klaus Paul Horn *Lisp (3rd Edition)* Addison-Wesley 1989 ISBN 0-201-08319-1
- Brian W. Kernighan and Rob Pike *The Unix Programming Environment* Prentice-Hall 1984 ISBN 0-13-937681-X
- Brian W. Kernighan and Dennis M. Ritchie *The C Programming Language (2nd Edition)* Prentice-Hall 1988 ISBN 0-13-110362-8
- Bjarne Stroustrup *The C++ Programming Language* Addison-Wesley 1991 ISBN 0-201-53992-6
- W. Richard Stevens *Advanced Programming in the Unix Environment* Addison-Wesley 1992 ISBN 0-201-56317-7
- W. Richard Stevens *Unix Network Programming* Prentice-Hall 1990 ISBN 0-13-949876-1

1. If you run it in the shell, you may get a core dump.
2. In case you did not know, a binary sort is an efficient way of sorting things into order and a bubble sort is not.
3. The reasons for this are buried in the mists of history.
4. Note, we did not use the `-o` flag to specify the executable name, so we will get an executable called `a.out`. Producing a debug version called `foobar` is left as an exercise for the reader!
5. They do not use the `MAKEFILE` form as block capitals are often used for documentation files like `README`.
6. Many Emacs users set their `EDITOR` environment to `emacsclient` so this happens every time they need to edit a file.

Last modified on: October 23, 2023 by [Mitchell Horne](#)

< [Prev](#)

[Home](#)

[Next](#) >



 English

System ▼



About

[FreeBSD](#)

[FreeBSD Foundation](#)

[Get FreeBSD](#)

[Code of Conduct](#)

[Security Advisories](#)

Documentation

[Documentation portal](#)

[Manual pages](#)

[Presentations and papers](#)

[Previous versions](#)

[4.4BSD Documents](#)

[Wiki](#)

Community

[Get involved](#)

[Community forum](#)

[Mailing lists](#)

[IRC Channels](#)

[Bug Tracker](#)

Legal

[Donations](#)

[Licensing](#)

[Privacy Policy](#)



Legal notices

© 1994-2023 The FreeBSD Project. All rights reserved

Made with  by the FreeBSD Community

